

Lab Number 1

TREVOR DEWALL

What did I learn?

This was certainly an interesting lab. It was very cool to learn how to analyze packets using a command line interface. I can see why using the command line interface would be very helpful. By using the proper filters it can display only the specific information you may be looking for. Now to be perfectly honest I did have a lot of difficulty with this lab because I have hardly ever used a command line packet analyzer before. I got a ton of errors some I was able to resolve others I was not able to resolve. I look forward to learning from my mistakes however as I progress in this course.

Question 1.1 - What kind of information does tcpdump display by default (i.e. only using the "-i eth0" flag)?

```
20:43:30.453728 IP 192.168.1.2.telnet > kali.41036: Flags [P.], seq 198:199, ack 79, win 4050, length 1
20:43:30.453790 IP kali.41036 > 192.168.1.2.telnet: Flags [.], ack 199, win 29200, length 0
20:43:30.595720 IP kali.41036 > 192.168.1.2.telnet: Flags [P.], seq 79:80, ack 199, win 29200, length 1
20:43:30.597825 IP 192.168.1.2.telnet > kali.41036: Flags [P.], seq 199:200, ack 80, win 4049, length 1
20:43:30.597841 IP kali.41036 > 192.168.1.2.telnet: Flags [.], ack 200, win 29200, length 0
20:43:30.688142 IP kali.41036 > 192.168.1.2.telnet: Flags [P.], seq 80:81, ack 200, win 29200, length 1
20:43:30.690293 IP 192.168.1.2.telnet > kali.41036: Flags [P.], seq 200:201, ack 81, win 4048, length 1
20:43:30.690309 IP kali.41036 > 192.168.1.2.telnet: Flags [.], ack 201, win 29200, length 0
20:43:30.779692 IP kali.41036 > 192.168.1.2.telnet: Flags [P.], seq 81:82, ack 201, win 29200, length 1
20:43:30.781831 IP 192.168.1.2.telnet > kali.41036: Flags [P.], seq 201:202, ack 82, win 4047, length 1
20:43:30.781895 IP kali.41036 > 192.168.1.2.telnet: Flags [.], ack 202, win 29200, length 0
20:43:30.967880 IP kali.41036 > 192.168.1.2.telnet: Flags [P.], seq 82:83, ack 202, win 29200, length 1
20:43:30.970147 IP 192.168.1.2.telnet > kali.41036: Flags [P.], seq 202:203, ack 83, win 4046, length 1
20:43:30.970162 IP kali.41036 > 192.168.1.2.telnet: Flags [.], ack 203, win 29200, length 0
20:43:31.039589 IP kali.41036 > 192.168.1.2.telnet: Flags [P.], seq 83:84, ack 203, win 29200, length 1
20:43:31.041839 IP 192.168.1.2.telnet > kali.41036: Flags [P.], seq 203:204, ack 84, win 4045, length 1
20:43:31.041903 IP kali.41036 > 192.168.1.2.telnet: Flags [.], ack 204, win 29200, length 0
20:43:31.121387 ARP, Request who-has gateway tell 192.168.1.138, length 46
20:43:31.131805 IP kali.41036 > 192.168.1.2.telnet: Flags [P.], seq 84:85, ack 204, win 29200, length 1
20:43:31.133952 IP 192.168.1.2.telnet > kali.41036: Flags [P.], seq 204:205, ack 85, win 4044, length 1
20:43:31.134018 IP kali.41036 > 192.168.1.2.telnet: Flags [.], ack 205, win 29200, length 0
20:43:31.199707 IP kali.41036 > 192.168.1.2.telnet: Flags [P.], seq 85:87, ack 205, win 29200, length 2
20:43:31.202102 IP 192.168.1.2.telnet > kali.41036: Flags [P.], seq 205:207, ack 87, win 4042, length 2
20:43:31.202166 IP kali.41036 > 192.168.1.2.telnet: Flags [.], ack 207, win 29200, length 0
20:43:31.213837 IP 192.168.1.2.telnet > kali.41036: Flags [P.], seq 207:217, ack 87, win 4042, length 10
```

Above is highlighted text that shows the default information that TCPDUMP displays by default. The information shown above is the time at which the packet was captured, The second piece of information it is showing is IP information or in this case the hostname of my kali box which is called just kali. It also shows the port it is using. It also shows the destination ip address 192.168.1.2 and the service the destination ip address is using which is telnet. The next piece of information that is shown are specific flags that were used like the push flag for instance. It then shows the sequence and acknowledgement numbers followed by the window size and then the length.

Question 1.2 - What kind of information can you gain from different flag combinations? Is there a flag that shows a hex representation of the packet's data contents? What about an ASCII representation? List all commands (including arguments) you used in your report, a description of the packet details provided by each command and what you saw as a result.

There is so much different information that can be showed from different flag combinations. Different flags allow you to pick and choose what information you want displayed.

An example of something I did was I did not like the default display of how TCPDUMP showed ip and port information so I used the -n option so it would show everything as ip addresses and it would list the port number next to the ip address. I also wanted to just filter the packets so only telnet displays. The command I used was
tcpdump -i eth0 port 23 -n

It then displayed the output like so.

```
21:44:50.758199 IP 192.168.1.131.41066 > 192.168.1.2.23: Flags [P.], seq 61:62, ack 164, win 29200, length 1
21:44:50.761816 IP 192.168.1.2.23 > 192.168.1.131.41066: Flags [P.], seq 164:165, ack 62, win 4067, length 1
21:44:50.761937 IP 192.168.1.131.41066 > 192.168.1.2.23: Flags [.], ack 165, win 29200, length 0
21:44:50.942148 IP 192.168.1.131.41066 > 192.168.1.2.23: Flags [P.], seq 62:63, ack 165, win 29200, length 1
21:44:50.944330 IP 192.168.1.2.23 > 192.168.1.131.41066: Flags [P.], seq 165:166, ack 63, win 4066, length 1
21:44:50.944443 IP 192.168.1.131.41066 > 192.168.1.2.23: Flags [.], ack 166, win 29200, length 0
21:44:51.210145 IP 192.168.1.131.41066 > 192.168.1.2.23: Flags [P.], seq 63:64, ack 166, win 29200, length 1
21:44:51.212457 IP 192.168.1.2.23 > 192.168.1.131.41066: Flags [P.], seq 166:167, ack 64, win 4065, length 1
21:44:51.212570 IP 192.168.1.131.41066 > 192.168.1.2.23: Flags [.], ack 167, win 29200, length 0
21:44:51.334014 IP 192.168.1.131.41066 > 192.168.1.2.23: Flags [P.], seq 64:65, ack 167, win 29200, length 1
```

As you can see the output by using flags is much cleaner than the default output TCPDUMP shows.

There is in fact a flag to show a hex representation of the packets data contents. The command I used was

tcpdump -i eth0 port 23 -n -xx (note I filtered by telnet to narrow down packets) -xx was the flag I used to show the hex representation of the data in the packet. Down below is the output I received.

```
21:56:53.246975 IP 192.168.1.131.41068 > 192.168.1.2.23: Flags [S], seq 81414773, win 29200, options [mss 1460,sackOK,TS val 2544101 ecr 0,nop,wscale 7], length 0
    0x0000: 0014 f2c4 c4c0 000c 292b ffc4 0800 4510
    0x0010: 003c 9b90 4000 4006 1b46 c0a8 0183 c0a8
    0x0020: 0102 a06c 0017 04da 4a75 0000 0000 a002
    0x0030: 7210 8404 0000 0204 05b4 0402 080a 0026
    0x0040: d1e5 0000 0000 0103 0307
21:56:53.250347 IP 192.168.1.2.23 > 192.168.1.131.41068: Flags [S.], seq 2991046403, ack 81414774, win 4128, options [mss 1460], length 0
    0x0000: 000c 292b ffc4 0014 f2c4 c4c0 0800 4510
    0x0010: 002c 9608 0000 ff06 a1dd c0a8 0102 c0a8
    0x0020: 0183 0017 a06c b247 bf03 04da 4a76 6012
    0x0030: 1020 a301 0000 0204 05b4 0000
21:56:53.250472 IP 192.168.1.131.41068 > 192.168.1.2.23: Flags [.]
```

There is also a flag that shows the data in ASCII representation. The command I used was `tcpdump -i eth0 -n -A port 23`

The -A flag allows you to view the ASCII Data. Down below is the output of some data.

```
22:02:38.771476 IP 192.168.1.2.23 > 192.168.1.131.41074: Flags [P.], seq 160:163, ack 40, win 4089, length 3
E..+w\.....r*>.K..O.P...}b....#...
22:02:38.771954 IP 192.168.1.131.41074 > 192.168.1.2.23: Flags [.] , ack 163, win 29200, length 0
E..(l.@.@.l.....r....O.*>.NP.r.....
```

Question 1.3 - How did you write the data to a pcap file out to the disk? Is there a way to stream the dump directly to a file, rather than buffering the packets prior to writing out? If so, how? (Include the command as well as any command line arguments you used to obtain this output.)

The command I used to write the data to a pcap file was `tcpdump -i eth0 -w filename.pcap`

This command wrote all data I captured during the dump to filename.pcap. You could then call that filename using `tcpdump -r filename.pcap` to view your captured data. I did not find a way to stream the dump directly to the file.

Question 2.4 - What flag did you use to load the pcap file? What happened when you ran the command?

The flag I used to load a pcap file was the -r flag. When I ran the command `tcpdump` loaded the pcap file and outputted it on the display for me.

Question 2.5 - Is there a way to obtain a list of individual TCP streams present in the capture? How?

Yes there is a way to obtain the TCP Streams present in the capture. What I did was I used the command `tcpdump -r capture2.pcap -z conv, tcp`. What this command did was it followed a TCP conversation that it found in the capture.

The output was show like so.

```
TCP Conversations
Filter:<No Filter>
```

		<-		->		Total		Relative		Duration	
		Frames	Bytes	Frames	Bytes	Frames	Bytes	Frames	Bytes	Start	
192.168.1.106:56907	<->	104.16.110.18:443	1	66	1	60	2	126	0.456337000	0.0225	
192.168.1.106:56903	<->	104.16.110.18:443	1	66	1	60	2	126	0.461366000	0.0221	
192.168.1.106:56904	<->	104.16.110.18:443	1	66	1	60	2	126	0.466354000	0.0223	
192.168.1.106:56905	<->	104.16.110.18:443	1	66	1	60	2	126	0.496219000	0.0230	
192.168.1.106:56906	<->	104.16.110.18:443	1	66	1	60	2	126	0.506267000	0.0225	
192.168.1.106:56902	<->	104.16.110.18:443	1	66	1	60	2	126	0.656407000	0.0225	
192.168.1.106:56900	<->	23.21.61.253:443	1	66	1	60	2	126	1.326404000	0.0308	

Although the problem is tshark does not do a very good job of constructing a tcp sequence.

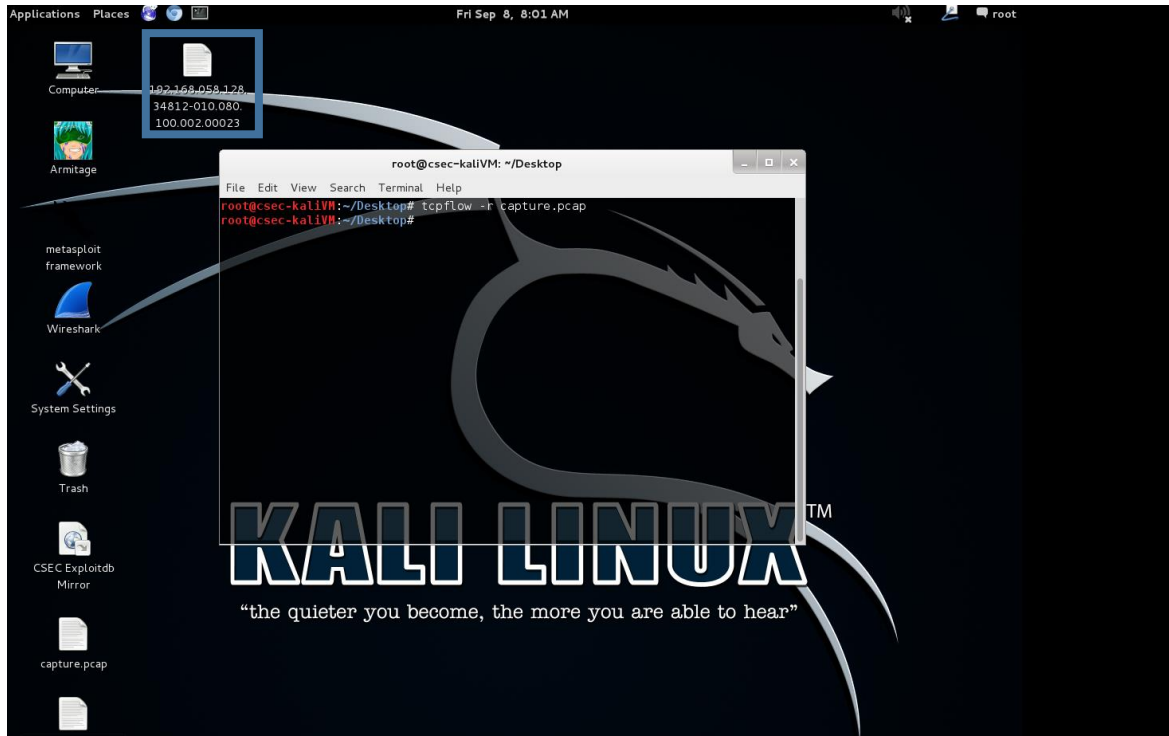
Question 2.6 - How do you apply a “display filter” to the output of a pcap file in TShark? How would you go about filtering a capture such that only packets being sent from a specific host might be seen?

For some reason I could not for life of me get this one to work. I attempted it on several different Kali boxes and always received the same errors. I googled the problem, used the man pages, etc. I tried several different combinations of the command as well. No matter what I could not get it to work. A push in the right direction would be awesome.

```
default report="fields"
use "-G ?" for more help
root@csec-kaliVM:~/Desktop# tshark -r capture.pcap -R -j DNS
tshark: Lua: Error during loading:
[string "/usr/share/wireshark/init.lua"]:46: dofile has been disabled due to ru
nning Wireshark as superuser. See http://wiki.wireshark.org/CaptureSetup/Capture
Privileges for help in running Wireshark as an unprivileged user
```

Question 2.7 What happened when you ran the above command? What kind of information can be gathered from a TCP stream?

When I ran the above command nothing happened in the terminal window however on the Desktop a file was created. I am not 100% sure what the file is for but that is what happened when I ran the command. In a TCP you can gather where the traffic was being sent from. It is handy to follow a TCP stream because you are able to follow the traffic from the beginning of the conversation to the end of the conversation which is really handy in forensics. How is this handy well take example of some employee who might not be doing what he is supposed to be doing an admin can sniff his traffic and follow TCP streams to find out where he has been going. See below screenshot for proof of what happened when I ran the command.



Question 3.8 At least two systems were engaged in a DNS exchange (client and server). List any two IP addresses involved in such an exchange and support with appropriate packets from the exchange. What were the IP addresses of the client and of the server? How did you find that information? (provide commands executed as well as a description of how each of the flags/arguments/commands affected the traffic.)

Two IP addresses I found in the exchange were 192.168.1.64 and it was making a DNS query to the DNS server which was 192.168.1.254.53. I found this out by filtering by DNS in TCP Dump. The command I used was `tcpdump -r nitroba.pcap -n port 53`. The flag I used was `-n` to make the output much neater cause the default tcpdump output is not very neat. I then told it to filter by DNS using port 53 which showed all of the DNS traffic.

```
oot@csec-kaliVM:~/Desktop# tcpdump -r nitroba.pcap -n port 53
Reading from file nitroba.pcap, link-type EN10MB (Ethernet)
1:51:07.376171 IP 192.168.1.64.34280 > 192.168.1.254.53: 25038+ A? www.google.com. (32)
1:51:07.378392 IP 192.168.1.64.57217 > 192.168.1.254.53: 29538+ AAAA? www.google.com. (32)
1:51:07.389299 IP 192.168.1.254.53 > 192.168.1.64.34280: 25038 5/8/6 CNAME www.l.google.com., A 74.125.19.103, A 74.125.19.147, A 74.125.19.104, A 74.125.19.99 (340)
1:51:07.390478 IP 192.168.1.254.53 > 192.168.1.64.57217: 29538- 0/0/0 (32)
1:51:07.404056 IP 192.168.1.64.44919 > 192.168.1.254.53: 63113+ AAAA? www.l.google.com. (34)
1:51:07.416518 IP 192.168.1.254.53 > 192.168.1.64.44919: 63113- 0/0/0 (34)
1:51:08.360749 IP 192.168.1.64.20128 > 192.168.1.254.53: 3217+ A? ssl.google-analytics.com. (42)
1:51:08.361958 IP 192.168.1.64.2319 > 192.168.1.254.53: 2112+ AAAA? ssl.google-analytics.com. (42)
```

(Sorry about the screenshot for some reason the lab machine I was using could not copy and paste between the bench machine and the virtual machine)

Question 3.9 The PCAP file features many HTTP sessions. List 5 individual domains to which the user connected, as shown in the PCAP, and explain how you found that information. Be sure to post the exact command(s) and descriptions of the argument(s) used

.

http sessions that I located in the capture were speakeasy.net, phishtank.com, google.com, luxgoddess.ccbn.net and save.ccbn.net. I did this by filtering the output with tcpdump using the command `tcpdump -r nitroba.pcap tcp port http`. This showed me all of the https sessions that occurred during this capture. Tcp port http filtered by http so it would show me all of the http sessions.

Question 3.10 Did you find anything else of interest in the PCAP file? If so, what did you find, and how did you find it?

I personally did not find anything else of interest in this file. This pcap file looked like normal traffic that occurs on a network which a lot of times is not interesting at all.